Getting an RTSP/RTMP Feed from a Hollyland Transmitter

Oct 19, 2026 · @Faini

The Hollyland Pyro H transmitter runs a standard RTSP video server on its own Wi-Fi hotspot, on port 5554. Any computer on that hotspot can play it in VLC or ffmpeg, or relay it out as RTMP. No hacking, firmware tools or reverse engineering needed.

Quick start: share it with a QR code

The script `holly-share.sh` does Steps 2 to 5 and the multi-viewer setup in one command, then shows a QR code that opens the stream on any phone. It runs on a Mac.

Set up once

Easiest: use the package. If you have `holly-share.zip`, unzip it, move the `holly-share` folder into your home folder, and double-click `Install.command`. It installs Homebrew and the tools for you. After that, double-click `Start Holly Share.command` whenever you want to share. If macOS says it could not verify a file, open System Settings → Privacy & Security and click **Open Anyway**.

No package? Do steps 1 to 3 by hand.

1. Install Homebrew, the free tool that installs everything else. Open Terminal (Cmd + Space, type Terminal, press Enter), paste this and follow the prompts. When it finishes, run the Next steps commands it prints, then open a new Terminal window. More at brew.sh.

```
/bin/bash -c "$(curl -fsSL
https://raw.githubusercontent.com/Homebrew/install/HEAD/install.sh)"
```

2. Install the three tools the script needs: ffmpeg (handles the video), MediaMTX (re-serves the stream) and qrencode (makes the QR codes).

```
brew install ffmpeg mediamtx qrencode
```

3. Save the script in a folder called `holly-share` in your home folder. Open the Script: holly-share.sh tab, copy everything in its box, then run this. It makes the folder, saves what you copied as `holly-share.sh` and lets it run:

```
mkdir -p ~/holly-share && pbpaste > ~/holly-share/holly-share.sh && chmod +x
~/holly-share/holly-share.sh
```

If someone sent you `holly-share.sh` as a file, move it into `~/holly-share` instead and run only `chmod +x ~/holly-share/holly-share.sh`.

Share the stream

1. Join the transmitter's Wi-Fi (Step 1).
2. In Terminal, run the command below. With the package, you can double-click Start Holly Share.command instead.
3. If macOS asks whether `mediamtx` may accept incoming connections, click **Allow**. Without it, phones can't connect.
4. A page with a QR code opens. Viewers scan it with their phone camera, then tap the speaker icon for sound.
5. To stop, click the Terminal window and press **Ctrl + C**.

```
~/holly-share/holly-share.sh
```

Change the page's text

The first time it runs, the script creates `share-page.html` next to itself in `~/holly-share`. That file holds the QR page's words and colours: change the heading, instructions or footer, save, and run the script again.

- Open it in any text editor. To find it, run `open ~/holly-share`. In TextEdit, first turn on Settings → Open and Save → Display HTML files as HTML code instead of formatted text.
- Keep the names in double curly braces, like `{{WATCH_QR}}` and `{{WATCH_URL}}`. The script swaps them for the QR codes and links.
- Delete the file to get the default page back.

What it does

- Finds the transmitter from your Wi-Fi's router address, then checks ports 5554, 554 and 8554 for RTSP. You only need nmap if all three fail.
- Starts MediaMTX, which pulls the stream from the transmitter once and re-serves it to every viewer.

- Makes a second copy with Opus audio. Browsers' low-delay playback (WebRTC) can't play the transmitter's AAC audio, so without it phones get picture but no sound.
- Picks the address viewers use: a wired connection if the Mac has one, otherwise the hotspot.
- Shows the QR code in Terminal and on the page from `share-page.html`, and saves it as a PNG to AirDrop or print.

Which Wi-Fi viewers join

Check the `Sharing on` line in Terminal. `en0` is usually the Mac's Wi-Fi: it means the Mac is only on the transmitter's hotspot, so phones must join it too. That hotspot is 2.4 GHz and each viewer adds about 3 Mbps, so keep it to a few people. Any other name, such as `en5` for a wired or USB network adapter, means phones join the normal Wi-Fi on that network.

To add a join-Wi-Fi QR code to the page, give the name and password of the network viewers should join:

```
~/holly-share/holly-share.sh --ssid "CAMERA-WIFI-NAME" --wifi-pass "PASSWORD"
```

Options

Option	What it does
`--ssid NAME --wifi-pass PASS`	Adds a join-Wi-Fi QR code as step 1 on the page
`--ip 192.168.218.10`	Sets the transmitter's address, if auto-detect fails
`--port 5554`	Sets the RTSP port, if it isn't 5554, 554 or 8554
`--share-ip 10.0.0.42`	Sets the address viewers use. A Tailscale address should let viewers outside the venue watch (not yet tested)
`--no-open`	Doesn't open the QR page

Addresses it serves

Use	Address
Phones and browsers (low delay, the QR code)	`http://<Mac IP>:8889/live`
Browser fallback (HLS, a few seconds behind)	`http://<Mac IP>:8888/cam`
VLC or OBS (RTSP)	`rtsp://<Mac IP>:8554/cam`
RTMP	`rtmp://<Mac IP>/cam`

What you need

- A Mac (Linux works the same way) with Terminal
- The transmitter powered on, with its Wi-Fi hotspot name and password (shown on the unit or in the HollyView app)
- Homebrew, to install the free tools: `nmap`, `ffmpeg`, `mediamtx` and `qrencode`
- About 15 minutes

Install the tools once:

```
brew install nmap ffmpeg mediamtx qrencode
```

Step 1: Join the transmitter's Wi-Fi

On your computer, connect to the transmitter's hotspot like any other Wi-Fi network. The hotspot is on 2.4 GHz. The 5.8 GHz signal is the private link to the receiver, so ignore it.

Step 2: Find the transmitter's IP address

The transmitter is the "router" of its own hotspot, so ask your Mac who handed it an address:

```
ipconfig getifaddr en0
ipconfig getoption en0 router
```

The first line is your computer's address. The second is the transmitter's. Ours was `192.168.218.10`. Save it in a variable so later commands can use it:

```
IP=192.168.218.10   # use your own router address
```

If `en0` returns nothing, run `networksetup -listallhardwareports` to find the name of your Wi-Fi port, or run `arp -a` to list every device on the hotspot.

Step 3: Find the open ports

The stream isn't on the usual RTSP port (554), so scan all 65,535 ports. It takes under a minute:

```
sudo nmap -p- -T4 --open $IP
```

Ours showed three open ports: `5554`, `11115` and `60007`.

Step 4: Find which port is the video

Ask nmap to identify each open port (swap in your own port numbers):

```
sudo nmap -sV -p 5554,11115,60007 $IP
```

The video port is the one listed as `rtsp`, which replied `RTSP/1.0 200 OK`. On ours that was **5554**. The other two are Hollyland's private control channels.

To confirm, ask the server to describe its stream:

```
printf 'DESCRIBE rtsp://%s:5554/live RTSP/1.0\r\nCSeq: 2\r\nAccept:
application/sdp\r\n\r\n' $IP | nc -w 2 $IP 5554
```

A working stream replies `200 OK` and lists `h264` video and `mpeg4-generic` audio (AAC, 48 kHz). The server ignores the path, so `/live` or anything else works.

Step 5: Watch the stream

In VLC: File → Open Network → `rtsp://192.168.218.10:5554/live` (use your own IP).

Or from Terminal, with low-latency settings:

```
ffplay -rtsp_transport tcp -fflags nobuffer -flags low_delay
rtsp://$IP:5554/live
```

Expect a second or two of black before the picture appears. The stream doesn't announce its video settings up front, so the player waits for the first keyframe.

Step 6: Send it out as RTMP

To push to YouTube, Twitch or any RTMP server, ffmpeg repackages the stream without re-encoding, so it adds no delay:

```
ffmpeg -rtsp_transport tcp -i rtsp://$IP:5554/live -c copy -f flv
rtmp://<server>/live/<stream-key>
```

If the server rejects the audio, replace `-c copy` with `-c:v copy -c:a aac -b:a 128k`.

Optional: Share it with several viewers

Small transmitters often allow only one or two RTSP connections. MediaMTX pulls the stream once and re-serves it to as many players as you need:

```
cat > ~/holly.yml <<'EOF'
paths:
  holly:
    source: rtsp://192.168.218.10:5554/live
    rtspTransport: tcp
EOF
mediamtx ~/holly.yml
```

Then connect to any of these:

Format	Address
RTSP	`rtsp://localhost:8554/holly`
RTMP	`rtmp://localhost/holly`
Browser (HLS)	`http://localhost:8888/holly`
Browser (WebRTC)	`http://localhost:8889/holly`

Other computers on the same network use your Mac's IP in place of `localhost`.

Check the stream quality

On the Pyro H the preview stream is full HD. The transmitter sets the quality, and the RTSP server has no settings to change.

	Pyro H (measured)
Resolution	1920 × 1080
Frame rate	30 fps
Video	H.264 High profile, about 3.1 Mbps
Audio	AAC-LC, 48 kHz stereo, about 100 kbps

The stream description advertises 256 kbps, but that number is a placeholder. To see the resolution, frame rate and codec on your unit:

```
ffprobe -v error -rtsp_transport tcp -show_entries
stream=codec_name,width,height,r_frame_rate,profile -of compact
rtsp://$IP:5554/live
```

To measure the real bitrate, record 10 seconds and read the totals:

```
ffmpeg -rtsp_transport tcp -i rtsp://$IP:5554/live -t 10 -c copy -f null -
2>&1 | tail -2
```

Divide the `video:` size by the `time=` value to get the bitrate. For example, 3,778 KiB in 10.06 seconds is 3,778 × 8 ÷ 10.06 ≈ 3,000 kbit/s, or about 3.1 Mbps.

At about 3 Mbps, 1080p looks fine for talking heads and slow shots, but fast motion shows some blockiness. Re-encoding can't add detail. For a cleaner picture, use an HDMI capture card on the receiver.

Optional: Re-encode the stream

Use `-c copy` whenever you can: it adds no delay and uses no CPU. Re-encode only to make the stream smaller, change its size or frame rate, or fix a server that rejects it.

```
ffmpeg -rtsp_transport tcp -i rtsp://$IP:5554/live \
  -vf "scale=1280:720:out_range=tv,fps=30" -pix_fmt yuv420p \
  -c:v libx264 -preset veryfast -tune zerolatency -b:v 3M -g 60 \
  -c:a aac -b:a 128k -f flv rtmp://<server>/live/<stream-key>
```

- `scale=1280:720` sets the output size. Remove it to keep 1080p.
- `out_range=tv` and `-pix_fmt yuv420p` convert the colours to the standard range, so players don't show washed-out or crushed blacks.
- `-b:v 3M` sets the video bitrate. Going above about 3 Mbps adds no detail.
- `-g 60` sends a keyframe every 2 seconds, which most streaming services expect.
- On Apple Silicon, replace `-c:v libx264 -preset veryfast -tune zerolatency` with `-c:v h264_videotoolbox` to use the hardware encoder.

Stop everything

In the Terminal window where it's running, press **Ctrl + C**. That stops ffplay, an ffmpeg push or MediaMTX.

If you closed the window, stop them from any Terminal:

```
pkill mediamtx
pkill ffmpeg
```

If you started MediaMTX as a background service with `brew services start mediamtx`, stop it with:

```
brew services stop mediamtx
```

To confirm nothing is still running (no output means it's all stopped):

```
pgrep -l mediamtx; pgrep -l ffmpeg
```

Troubleshooting

Problem	Fix
Black screen or no picture	Swap `-rtsp_transport tcp` for `-rtsp_transport udp`, and wait a few seconds for a keyframe
`deprecated pixel format used, make sure you did set range correctly`	Harmless. The stream labels its colours the old way (`yuvj420p`). Add `-loglevel error` to hide it. When re-encoding, add `-vf scale=out_range=tv -pix_fmt yuv420p`
`command not found: nc`, `grep` or `head` after running a loop	In zsh, never name a loop variable `path`; it overwrites your command search path. Open a new Terminal tab to fix it
nmap shows no RTSP port	Your model may use a different port. Run `sudo nmap -sV` on every open port nmap found
Stream drops when a second viewer joins	Use holly-share.sh or MediaMTX (above) so the transmitter only serves one connection
Picture breaks up	The 2.4 GHz hotspot is carrying about 3 Mbps. Move the computer closer to the transmitter
IP address changed	Re-run Step 2; the hotspot can hand out a different address after a restart
Phones can't open the QR link	Put the phone on the same network as the Sharing on address. Allow mediamtx in System Settings → Network → Firewall
Picture on the phone but no sound	Browsers start muted. Tap the speaker icon
Port 8554 is already in use	MediaMTX is already running. Stop it with brew services stop mediamtx and pkill mediamtx, then run the script again
command not found: brew	Run the Next steps commands Homebrew printed at the end of its install, then open a new Terminal window

Problem	Fix
permission denied when starting the script	Run chmod +x ~/holly-share/holly-share.sh, then start it again
Missing: ffmpeg, mediamtx or qrencode	Install what it lists with brew install, as in Set up once

Tested on a Hollyland Pyro H in October 2026. Other Hollyland models may use different ports, but the same steps apply.